Zonoth

SESSION 06 · 2H

Workloads: Deployments, rollouts and probes

Stop running Pods by hand. Let a controller own them, ship 1.0 → 1.1 with nobody noticing, and undo it when it goes wrong.

Labels & selectors | Deployments & rollouts | Probes | Other controllers | Lab 06

SESSION 06

What we'll cover today

1 · Labels, selectors, controllers

How Kubernetes finds objects, the recommended `app.kubernetes.io/*` labels, and ReplicaSet vs Deployment.

2 · Rollouts & probes

`maxSurge` / `maxUnavailable`, `rollout status/history/undo`, and the three probes that make it safe.

3 · Other controllers

DaemonSet, StatefulSet, Job and CronJob at a glance — plus scaling and graceful termination.

4 · Lab 06

Make the Pinboard API resilient: 3 replicas, probes, a zero-downtime 1.0 → 1.1, then a rollback from a broken image.

01

Labels, selectors, controllers

Kubernetes has no folders. Objects find each other by matching key/value labels — and that is the whole trick.

Labels — and what they are not

- **Labels** are identifying key/value pairs used to **select** objects
- **Annotations** are non-identifying metadata — never selected on (checksums, URLs, tool state)
- Services, ReplicaSets, NetworkPolicies and HPAs all work by selecting labels
- Labels are mutable; change one and membership changes instantly

YAML

```yaml
metadata:
  name: pinboard-api
  namespace: pinboard
  labels:
    app.kubernetes.io/name: pinboard-api
    app.kubernetes.io/component: api
    app.kubernetes.io/part-of: pinboard
  annotations:
    kubernetes.io/change-cause: "bump to 1.1"
```

⚠ Relabel a running Pod and its ReplicaSet will consider it lost — and immediately create a replacement.

SYNTAX

Label keys: `[prefix/]name`

- **name** and **value**: up to 63 characters — letters, digits, `-` `_` `.` (a value may be empty)
- **prefix** (optional): a DNS subdomain — `app.kubernetes.io`, `acme.com`; `kubernetes.io/` is reserved

YAML

```yaml
labels:                          # on the object
  app.kubernetes.io/name: pinboard-api
  app.kubernetes.io/part-of: pinboard
  tier: back                     # no prefix: yours
```

YAML

```yaml
selector:                        # in the Deployment
  matchLabels:
    app.kubernetes.io/name: pinboard-api
```

Short keys for quick labels you add and remove yourself. **Full prefixed keys in selectors**: a selector is a contract — a prefixed key cannot collide with another tool's `name`, and every tool understands it.

MATCHING

Selectors: equality and set-based

BASH

```
kubectl get pods -l app.kubernetes.io/name=pinboard-api
kubectl get pods -l 'app.kubernetes.io/component!=db'
kubectl get pods -l 'app.kubernetes.io/component in (api,web)'
kubectl get pods -l app.kubernetes.io/part-of=pinboard --show-labels
kubectl delete pods -l app.kubernetes.io/component=api    # careful
```

YAML

```
selector:                       # Deployments/ReplicaSets: matchLabels + matchExpressions
  matchLabels:
    app.kubernetes.io/name: pinboard-api
  matchExpressions:
    - { key: app.kubernetes.io/component, operator: In, values: [api] }
```

CONVENTION

The recommended labels

LABEL	PINBOARD VALUE	WHY
app.kubernetes.io/name	`pinboard-api`	The application itself — our selector key
app.kubernetes.io/component	`api` · `web` · `db`	The role inside the system
app.kubernetes.io/part-of	`pinboard`	Groups the whole app — one query shows everything
app.kubernetes.io/version	`"1.1"`	Human-readable release; not the selector
app.kubernetes.io/managed-by	`kubectl` · `helm`	Who owns this object

Select only on **stable** labels. Never put `version` in `matchLabels` — the selector is immutable, so every release would need a new Deployment.

ReplicaSet: keep N Pods alive

- Watches Pods matching its selector, counts them, creates or deletes to reach `replicas`
- Owns its Pods through `metadata.ownerReferences`
- Delete a Pod → a replacement appears in seconds
- Node dies → its Pods are recreated **elsewhere**
- It cannot change the Pod template safely — no rollout logic at all



The self-healing you were promised

This is the Session 05 reconciliation loop with one concrete goal: **observed replicas == desired replicas**.

You will almost never write a ReplicaSet by hand — but every `kubectl get rs` line you see for the rest of the course is one.

LAYERING

Deployment = ReplicaSets over time

01
Deployment
You edit this — the desired template

→

02
ReplicaSet 1.0
Scaled 3 → 0 during the rollout

→

03
ReplicaSet 1.1
New template hash, scaled 0 → 3

→

04
Pods
Owned by whichever RS created them

	REPLICASET	DEPLOYMENT
Keeps N Pods running	yes	yes — through a ReplicaSet
Change the image safely	no	rolling update, one step at a time
History & rollback	no	`rollout history` / `undo`

THE MANIFEST

The Pinboard API Deployment

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pinboard-api
  namespace: pinboard
spec:
  replicas: 3
  selector:
    matchLabels:                          # immutable — choose it once
      app.kubernetes.io/name: pinboard-api
  template:                               # ← the Pod from Session 05
    metadata:
      labels:
        app.kubernetes.io/name: pinboard-api
```

- `template.spec` is the Pod spec you already wrote — containers, resources, probes
- The template's labels **must** satisfy `spec.selector`, or the API rejects it

02

Rollouts

Shipping `pinboard-api:1.0` → `1.1` while people are using it — and getting back if you were wrong.

MECHANICS

How a rolling update actually steps

01 — **3 × 1.0**
Steady state, all Ready

→

02 — **+1 surge**
One 1.1 Pod created — 4 exist

→

03 — **wait Ready**
Readiness probe passes, endpoint added

→

04 — **−1 old**
An old Pod is terminated gracefully

↻

05 — **3 × 1.1**
Repeat until the new RS owns all replicas

YAML

```yaml
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # at most 4 Pods exist at once
      maxUnavailable: 0    # never fewer than 3 Ready — capacity is never reduced
```

CHOOSING

RollingUpdate vs Recreate

RollingUpdate (default)

- Zero downtime when readiness is honest
- Old and new versions run **at the same time**
- Needs backward-compatible APIs and DB schemas
- What Pinboard uses

Recreate

- Kill everything, then start the new version
- Guaranteed downtime — plan a window
- Right when versions genuinely cannot coexist: a breaking migration, or a single `ReadWriteOnce` volume

⚠ Rolling updates mean N and N+1 serve traffic together for a minute. Your migrations must be additive — expand, deploy, contract.

DRIVING IT

status · history · undo

BASH

```
kubectl set image deployment/pinboard-api api=pinboard-api:1.1 -n pinboard
kubectl rollout status deployment/pinboard-api -n pinboard   # blocks until done
kubectl get rs -n pinboard                                   # old RS at 0, new at 3

kubectl rollout history deployment/pinboard-api -n pinboard
# REVISION  CHANGE-CAUSE
# 1         initial 1.0
# 2         bump to 1.1

kubectl rollout undo deployment/pinboard-api -n pinboard
kubectl rollout undo deployment/pinboard-api --to-revision=1 -n pinboard
kubectl rollout pause deployment/pinboard-api -n pinboard    # edit safely, then resume
```

`revisionHistoryLimit: 5` keeps five old ReplicaSets — that is exactly how many revisions you can roll back to.

LIVE DEMO · 8 MIN

Ship 1.0 → 1.1 without dropping a request

Three replicas of `pinboard-api:1.0`, a traffic loop on `/api/info`, and a rolling update that flips the theme from **emerald** to **amber** one Pod at a time.

What you'll see · Deployment → RS → Pod, surge Pod, theme & hostname rotating, zero errors, `rollout history`

Files · demos/session06/api-deploy-v1.yaml · api-deploy-v2.yaml

Duration · 8 min

03

Probes

The cluster cannot read your logs. Probes are the only way it knows whether your container is alive and whether it is ready.

THREE JOBS, THREE PROBES

liveness · readiness · startup

livenessProbe

Am I broken? On failure the kubelet **restarts the container**. For deadlocks and wedged processes only.

readinessProbe

Should I get traffic? On failure the Pod is **removed from Service endpoints**. Nothing is restarted.

startupProbe

Have I booted yet? Holds the other two off until it passes — for slow starters, without loosening liveness.

✓ Handlers: `httpGet` · `tcpSocket` · `exec` · `grpc`. Pinboard exposes `/healthz` (liveness) and `/readyz` (readiness, 503 when the store is unreachable).

CONFIGURATION

Probes on the Pinboard container

- `periodSeconds` × `failureThreshold` = time to react
- Readiness here reacts in ~15 s; liveness in ~30 s
- A `startupProbe` of 2 s × 30 gives 60 s to boot
- Prefer a startup probe over a long `initialDelaySeconds`
- `port: http` refers to the **named** container port

YAML

```yaml
readinessProbe:          # drop from endpoints
  httpGet:
    path: /readyz
    port: http
  periodSeconds: 5
  failureThreshold: 3
livenessProbe:           # restart the container
  httpGet:
    path: /healthz
    port: http
  periodSeconds: 10
  failureThreshold: 3
```

WHY THIS SESSION EXISTS

A rolling update without readiness is an outage

No readinessProbe

"Ready" means **the process started**. Kubernetes adds the Pod to the Service the instant the container runs — before the HTTP listener is up.

TEXT

```
1.0 emerald  api-7f4c
curl: (52) Empty reply from server
curl: (7) Connection refused
1.1 amber    api-9d21
```

With readinessProbe

The rollout waits for `/readyz` to answer 200 before it removes an old Pod. With `maxUnavailable: 0`, capacity never dips.

TEXT

```
1.0 emerald  api-7f4c
1.0 emerald  api-7f4c
1.1 amber    api-9d21
1.1 amber    api-9d21
```

⚠ The failure is silent in `kubectl`: the rollout reports success either way. Only the users see it.

GETTING IT WRONG

Probe pitfalls

- **Liveness that checks dependencies** — a DB blip restarts every replica at once
- **Thresholds too tight** — a GC pause or a slow disk becomes a restart loop
- **Liveness but no readiness** — traffic hits a Pod that is up but not warm
- **An expensive endpoint** — a probe every 5 s × 200 Pods is real load; keep it cheap and local
- **No startup probe on a slow starter** — liveness kills it mid-boot, forever

Sane default: **always** a readiness probe. Add liveness only when you can name the failure it recovers from.

LIVE DEMO · 6 MIN

Break it, watch it, undo it

Roll out `pinboard-api:1.2-broken` and watch `CrashLoopBackOff` stall it while `maxUnavailable: 0` keeps the old Pods serving — then get out with `rollout undo`.

What you'll see · a stuck rollout that never took the site down

Files · demos/session06/api-deploy-broken.yaml

Duration · 6 min

04

Beyond Deployments

Four more controllers, one scaling command, and the ten seconds your Pod gets to say goodbye.

PICK THE RIGHT ONE

Controllers at a glance

Deployment

Stateless, interchangeable replicas. Random names, rolling updates, any Pod can serve any request. **pinboard-api, pinboard-web.**

DaemonSet

One Pod **per node**, automatically on new nodes. Log collectors, CNI agents, node exporters. **kube-proxy is one.**

StatefulSet

Stable names (`pinboard-db-0`), stable storage per Pod, ordered start and rollout. **Session 08.**

Job & CronJob

Run to completion, once or on a schedule. Migrations, backups, batch imports — not servers.

RUN TO COMPLETION

Jobs and CronJobs for Pinboard

- `restartPolicy` must be `OnFailure` or `Never` — never `Always`
- `backoffLimit` caps retries; `ttlSecondsAfterFinished` cleans finished Pods up
- CronJob wraps a Job template: `schedule: "* * * * *"`, plus `concurrencyPolicy: Forbid`
- Schedules use the cluster's timezone unless you set `timeZone`

YAML

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: pinboard-migrate
  namespace: pinboard
spec:
  backoffLimit: 2
  ttlSecondsAfterFinished: 300
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: postgres:17-alpine
```

EVERYDAY OPERATIONS

Scaling and restarting

BASH

```
kubectl scale deployment/pinboard-api --replicas=5 -n pinboard
kubectl get deploy pinboard-api -n pinboard
# NAME           READY   UP-TO-DATE   AVAILABLE   AGE
# pinboard-api   5/5     5            5           12m

kubectl rollout restart deployment/pinboard-api -n pinboard   # new ConfigMap? restart
kubectl scale deployment/pinboard-api --replicas=3 -n pinboard
```

rollout restart

Stamps a new annotation on the Pod template, so a normal rolling update replaces every Pod — safely, one at a time. The way to pick up a changed ConfigMap or Secret.

Careful with drift

`scale` changes the live object, not your YAML. The next `apply` puts `replicas` back — unless an HPA owns it (Session 09).

SHUTTING DOWN

terminationGracePeriodSeconds

1. Pod marked Terminating — **removed from endpoints**
2. `preStop` hook runs, if any
3. **SIGTERM** to PID 1 in each container
4. Grace period counts down (default 30 s)
5. Still alive? **SIGKILL** — no cleanup, dropped connections

⚠ Steps 1 and 3 are concurrent. A short `preStop: sleep 5` lets endpoint removal propagate before you stop accepting requests.

YAML

```yaml
spec:
  terminationGracePeriodSeconds: 15
  containers:
    - name: api
      image: pinboard-api:1.1
      # SHUTDOWN_GRACE=10s — the API drains
      # in-flight requests, then exits 0
```

✓ Session 03's exec-form lesson pays off here: shell form means PID 1 is `/bin/sh` and SIGTERM never reaches your process.

LAB 06 · 30–40 MIN

Make the Pinboard API resilient

Turn last session's single Pod into a Deployment with three replicas, probes and resources. Then ship `1.0` → `1.1` under a `curl` loop, break it with `1.2-broken`, and roll back.

You'll practise: writing a Deployment · `rollout status` · watching theme `emerald` → `amber` · `CrashLoopBackOff` · `rollout undo` · `kubectl scale` 3 → 5 → 3 · a migration Job and a CronJob heartbeat

Key takeaways

✓ Labels are the wiring

Selectors are how controllers and Services find Pods. Use the `app.kubernetes.io/*` set and never select on version.

✓ A Deployment plays ReplicaSets

One per template hash — which is why `maxSurge` / `maxUnavailable` tune the rollout and `undo` takes seconds.

✓ Readiness is the contract

Without it a rolling update is an outage. Liveness restarts, readiness routes, startup buys boot time.

✓ Next up

Session 07 — Services, cluster DNS and Ingress: how anyone reaches these Pods in the first place.

Zonoth

Questions?

Up next — Session 07: Networking — Services, DNS, Ingress. Keep the Deployment running.